Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Bu ders öğrenci proje sunumlarına ayrılmıştır. Üç sunum yapıldı: k-boyutlu ağaçlar (k-d tree), standart hash fonksiyonlarının saldırılara karşı karşılaştırılması ve blokzincir (blockchain) için algoritmalar ve hash fonksiyonları. Aşağıda yalnız sunumların akademik içeriği ve hocanın akademik yorumları yer alır.

Genel Konular

- k-boyutlu ağaçlar (k-d tree) ve güncel kullanımları (birinci sunum)
  - Tek boyutlu veride en bilinen arama yapısı ikili arama ağacıdır: küçük elemanlar sola, büyükler sağa yerleşir. Gerçek dünyadaki veri çoğunlukla çok boyutludur; k-d ağacı bu durum için önerilmiştir. Adındaki "d", boyut (dimension) anlamındadır.
  - Algoritma 1970'lerde J. L. Bentley tarafından önerilmiş, 1990'da hesaplamalı geometri konferansında (SoCG) yine onun tarafından sunulmuştur. Amacı en yakın komşuyu (nearest neighbour) bulmak, aranan değeri bulmak ve aralık sorgusu (range query) yapmaktır.
  - Kullanım alanları: veritabanı yönetim sistemleri, haritalandırma, meteoroloji ve oşinografi, ağ (network) uygulamaları.
  - Literatür taramasında altı çalışma incelendi (ekrandan, 00:30:00): yapılandırılmamış ızgara (unstructured grid) yeniden eşlemede kaba kuvvet ile k-d ağacı karşılaştırması (2020), en yakın komşu araması için k-d ağacını yeniden ele alan bir çalışma (2019), SDN tabanlı araç ağlarında akış sınıflandırması (2021), şifreli zaman serisi verisi üzerinde gizliliği koruyan benzerlik aralığı sorgusu (2022), kablosuz sensör ağlarında k-d ağacına dayalı kümeleme yönlendirme protokolü (2018), uyarlanır yarıçaplı (adaptive radius) k-d ağacıyla gerçek zamanlı akan veri kümeleme (2020).
  - Paket sınıflandırma çalışmasında geometrik uzay da kullanılarak klasik k-d ağacına göre 24 kata kadar hızlanma bildirildiği aktarıldı.
- k-d ağacının yapısı (ekrandan, 00:13:30)
  - Düğümler (iç düğümler) yalnız boyutlara göre kırılım (bölme) noktalarını tutar; yaprak düğümler verinin kendisini tutar. Veriye ancak düğümlerdeki arama bitince, yaprakta ulaşılır.
  - Bazı ikili arama ağacı gerçeklemelerinde veri iç düğümde de tutulur; bu yapıda tutulmaz. İç düğümlerden geçen yol bir indekstir.
  - İki boyutlu örnekte ($k = 2$) noktalar sırayla birinci ve ikinci boyuta göre sola ya da sağa ayrılır; yöntem k boyuta genelleşir. Ağaç her zaman dengeli olmaz; bazı yapraklara daha uzun yoldan ulaşılır (ekrandan, 00:18:30).
- Bellek içi ve bellek dışı erişim karmaşıklığı (ekrandan, 00:16:25)
  - Bellek içi erişim: ağaç boyunca izlenen yol. Bellek dışı erişim: disk erişimi ya da ağdan alınan/gönderilen paket sayısı gibi işlemler. Bellek dışı erişim çok daha pahalı olduğu için karmaşıklık asıl buna göre hesaplanır.
  - Ağaca eleman eklemenin bellek içi maliyeti, ikili arama ağacındaki arama maliyetiyle aynıdır: $O(\log n)$.
  - Bir yaprak (veri sayfası) tek bir kayıt değil, bir disk sayfasına sığan kadar kayıt tutar; bu sayı m'dir. Arama ve eklemenin bellek dışı maliyeti $O(\log(n/m))$'dir.
  - Yaprakta yer yoksa yeni yaprak açmak için bir, son düğümü düzenlemek için bir bellek dışı erişim daha gerekir: $O(2 + \log(n/m))$. $\log(n/m)$ küçük kaldığı için sunumda endüstride bu maliyetin genelde yaklaşık 2 disk erişimi sayıldığı söylendi.
- Aralık sorgusu ve disk düzeni (ekrandan, 00:20:00)

  - Sunumdaki iddia: veri sayfaları diskte aynı iz (track) üzerinde yan yana tutulursa, bir eleman bulunduktan sonra komşu elemanlar yan sayfalardan okunur; bu yüzden aralık sorgusunun maliyeti tek bir elemanı bulmakla aynı sayılabilir.
  - Slayttaki formül: Transfer Time $= \dfrac{\text{Number of Bytes Transferred}}{\text{Number of Bytes on a Track}} \times$ Rotation Time. Sunumda, ardışık okumada dönme süresinin sıfır kabul edilebileceği söylendi.
- Prototip uygulama (ekrandan, 00:23:05 ve 00:25:10)
  - C ile iki boyutlu bir k-d ağacı yazıldı. Veri: Türkiye'deki 81 il merkezinin enlem (latitude) ve boylam (longitude) değerleri ve birkaç ek alan (plaka, ad, yüzölçümü, nüfus, nüfus yoğunluğu, bölge).
  - İşlevler: ağaç oluşturma, kayıt ekleme, kayıt arama (enlem ve boylamla en yakın kayıt), aralık sorgusu (veri sayfalarını yazdırma), disk sayfa boyutunu değiştirme, indeksi diske yazma.
  - Düğüm yapısı: `bool leftLeafPtr, rightLeafPtr, isLat; double data; int leftPage, rightPage; struct Node *left, *right, *parent;`. Şehir kaydı 112 bayt yer kaplar; 1 KB sayfa seçilince bir sayfaya $\lfloor 1024/112 \rfloor = 9$ il sığar.
- Hash fonksiyonlarının saldırılara karşı karşılaştırılması (ikinci sunum, İngilizce)
  - Kriptografik hash fonksiyonundan beklenen özellikler: çarpışma saldırısına direnç (iki farklı M, M' için aynı $H(M)$ bulunamamalı), ön görüntü (preimage) saldırısına direnç (verilen $H(M)$'den M bulunamamalı) (ekrandan, 00:32:20).
  - İncelenen algoritmalar: MD5 (Merkle-Damgård, 128 bit çıktı), RIPEMD-160 (160 bit), SHA-1 (160 bit), SHA-2 (256/384/512 bit), SHA-3 (Keccak, sünger/sponge yapısı) (ekrandan, 00:35:00, 00:37:00, 00:40:00).
  - Genel (jenerik) çarpışma saldırısı doğum günü paradoksuna dayanır: n bitlik çıktı için yaklaşık $2^{n/2}$ işlem. Hızlı çarpışma saldırıları algoritmanın tasarım zayıflıklarını kullanarak bu sınırın altına iner.
  - Serbest başlangıçlı (freestart) çarpışma, hash fonksiyonunun kırıldığı anlamına gelmez; daha fazla analiz gerektiğini gösterir.
  - Saldırı karmaşıklıkları tablosu (ekrandan, 00:41:10): MD5 çarpışma 2^{39}, hızlı çarpışma 2^{18}; RIPEMD-160 çarpışma 2^{67}, ön görüntü $2^{158.91}$; SHA-1 çarpışma $< 2^{69}$ ve 2^{61}, freestart çarpışma; SHA-256 ön görüntü $2^{255.5}$, SHA-512 ön görüntü $2^{511.2}$; SHA-3 için pratik çarpışma ve yakın çarpışma (near-collision) sonuçları.
- Blokzincir için algoritmalar ve hash fonksiyonları (üçüncü sunum)
  - Dağıtık defter (distributed ledger): sürekli büyüyen, kopyası herkeste bulunan kayıt yapısı. Merkezi yapıda merkez çökerse sistem çöker (tek hata noktası, single point of failure); merkeziyetsiz yapıda bir düğümün çökmesi sistemi durdurmaz. Ağın %51'inden fazlası kötü niyetliyse ağın kontrolü ele geçirilebilir.
  - Blok yapısı (ekrandan, 01:00:00): başlıkta (header) `Hash(Prev)`, `Nonce`, `Difficulty`, `Timestamp`, `Merkle Root`; gövdede işlemler (transactions). `Hash(Prev)` önceki bloğun başlığının hash'idir; bloklar böylece zincirlenir.
  - Merkle ağacı (ekrandan, 00:56:30): her işlemin hash'i alınır ($H_1..H_4$), ikişer birleştirilip yeniden hash'lenir (H_{12}, H_{34}), sonra kök (H_{1234}) bulunur. Bir işlem değişirse köke kadar bütün yoldaki hash'ler değişir.
  - Konsensüs mekanizmaları: Proof of Work, Proof of Stake, Delegated Proof of Stake. Proof of Work'te madenci (miner), başlığın SHA-256 hash'i baştan belirli sayıda sıfırla başlayana kadar nonce değerini artırarak dener.
  - Dijital imza (ekrandan, 01:02:00): kullanıcı işlemin hash'ini özel anahtarıyla şifreler; madenci imzayı açık anahtarla çözer, işlemin hash'ini kendisi hesaplar ve iki hash'i karşılaştırır.
  - Akıllı sözleşmeler: Ethereum ağında çalışan kod; Ethereum bytecode'una çevrilip Ethereum sanal makinesinde (EVM) çalışır.

Hocanın Özellikle Vurguladığı Kısımlar

- k-d ağacının güncel kullanımları
  - Hoca, k-d ağacının temel bir yapı olduğunu ve matematiksel arka planının sunumda anlatıldığını söyledi. Kendisini asıl ilgilendiren kısmın güncel kullanımlar ve k-d ağacının başka algoritmalarla nasıl birleştirildiği olduğunu belirtti.
- "Eski bir algoritma ama hâlâ güncelleniyor" ifadesi

  - Hoca bu ifadeyi sorguladı. Sunum yapan öğrenci ifadeyi şöyle düzeltti: temel algoritma aynıdır; yeni çalışmalar ona kendi alanlarına göre eklemeler yaparak zenginleştirir.
- Veri setinin kaynağı
  - Hoca, hem literatürdeki son çalışmada hem prototipte hangi verinin kullanıldığını sordu. "Sentetik" kelimesinin rastgele üretilmiş veri anlamına gelip gelmediğini netleştirdi: prototipteki veri, il merkezlerinin gerçek koordinatlarından derlenmişti, rastgele değildi.

Kısa Tekrar Notları

- k-d ağacı: iç düğümler bölme değerini ve bölme boyutunu tutar, veri yalnız yapraklardadır.
- Ekleme ve aramanın bellek içi maliyeti $O(\log n)$, bellek dışı maliyeti $O(\log(n/m))$; yaprak doluysa $O(2 + \log(n/m))$. m: bir sayfaya sığan kayıt sayısı.
- Disk tabanlı yapılarda karmaşıklık, pahalı olan bellek dışı erişime göre ölçülür.
- Prototip: 112 baytlık şehir kaydı, 1 KB sayfa, sayfa başına 9 kayıt.
- Hash fonksiyonu güvenliği: çarpışma direnci, ön görüntü direnci. Jenerik çarpışma $2^{n/2}$ (doğum günü paradoksu).
- MD5 ve SHA-1 pratikte zayıf; SHA-2 ve SHA-3 için bilinen saldırılar kaba kuvvete çok yakın ya da azaltılmış tur sayısı içindir.
- Blokzincir: `Hash(Prev)` ile zincirleme, Merkle kökü ile işlem bütünlüğü, nonce ile Proof of Work, imza ile işlem doğrulama.

Detaylı Açıklamalar

- k-d ağacında arama ve ekleme
  - Arama kökten başlar. Her iç düğümde bölme boyutu (prototipte `isLat`: enlem mi boylam mı) ve bölme değeri (`data`) karşılaştırılır; aranan değer küçükse sola, büyükse sağa gidilir. İç düğümlerde hiçbir veri kaydıyla karşılaşılmaz. Yaprağa ulaşıldığında o yaprağın veri sayfası diskten okunur.
  - Ekleme de aynı yolu izler ve kayıt ulaşılan yaprağa yazılır. Yaprakta yer varsa ek maliyet yoktur. Yer yoksa yeni bir yaprak açılır ve yaprağın bağlı olduğu düğüm güncellenir; bu iki ek disk erişimidir. Bu yüzden toplam maliyet $O(2 + \log(n/m))$ olur.
  - Örnek: disk sayfası 4 KB ve bir kayıt bunun onda biri büyüklüğündeyse bir yaprağa 10 kayıt sığar, yani $m = 10$. Prototipte kayıt 112 bayttır ve 1 KB sayfa seçilince $m = \lfloor 1024/112 \rfloor = 9$ olur (ekrandan, 00:25:10). Demoda birbirine yakın iller aynı veri sayfasına düştü; örneğin bir sayfada Samsun, Ordu, Rize, Artvin, Kars, Trabzon ve Ardahan birlikteydi.
  - Demoda Ordu'nun koordinatları girilerek arama yapıldı ve aynı veri sayfasındaki Samsun kaydı ayrıntılarıyla bulundu.
- Aralık sorgusu
  - Sunumda, yaprakların soldan sağa okunduğunda veriyi sıralı verdiği ve veri sayfaları diskte yan yana tutulursa bir aralığın tek bir sayfayı bulma maliyetiyle okunabileceği söylendi.
  - **Ders sonrası not:** Sunumda "k-d ağacı elemanları sıralı tutmayı garanti eder" ve "aralık sorgusu tek eleman bulmakla aynı maliyettedir" dendi. Çok boyutlu veride tek bir sıralama yoktur; yaprak sırası noktaları uzaysal olarak gruplar, ama bir aralık sorgusu genelde birden fazla dala inmek zorunda kalır. Dengeli bir k-boyutlu k-d ağacında dikdörtgen aralık sorgusunun en kötü durumu büyük olasılıkla $O(n^{1-1/k} + r)$'dir (r: dönen kayıt sayısı). Ardışık sayfa okumanın ucuz olduğu doğrudur, ama "aynı maliyet" iddiası yalnız tek dala düşen küçük aralıklar için geçerlidir.
- Hash fonksiyonlarının yapısı
  - MD5, RIPEMD-160, SHA-1 ve SHA-2 Merkle-Damgård yapısını kullanır; SHA-3 sünger (sponge) yapısını kullanır. Slayttaki karşılaştırma tablosunda (ekrandan, 00:40:00) blok boyutu MD5, RIPEMD-160 ve SHA-1 için 512 bit, SHA-2 için 512/1024 bit, SHA-3 için 1088/576 bit; kullanılan işlemler ADD, XOR, AND, OR, NOT, SHIFT ve ROTATE'tir.

  - **Ders sonrası not:** Slayt tablosunda MD5 için tur (rounds) sayısı 18, SHA-3 için kelime boyutu 320 bit ve çıktı boyutu 1600 bit yazıyor. MD5'te büyük olasılıkla 4 tur × 16 adım = 64 adım vardır. SHA-3'te 1600 bit iç durumun (state) boyutudur; kelime (lane) 64 bit, çıktı ise 256/512 bittir. Slayttaki "256, 348 ve 512 bit" ifadesindeki 348 de 384 olmalıdır (SHA-384).
  - Sunumda SHA-1'in 2011'de standarttan çıkarıldığı ve bu kararı NSA'nın verdiği söylendi.
  - **Ders sonrası not:** SHA-1'i kullanımdan kaldıran kurum büyük olasılıkla NSA değil NIST'tir (2011'de kullanımdan kaldırma kararı).
- Blokzincirde Proof of Work
  - Madenci başlığa bir nonce yazar ve başlığın SHA-256 hash'ini hesaplar. Hash'in baştaki belirli sayıda hanesi sıfır değilse nonce'u bir artırır ve yeniden dener. Her denemenin çıktısı rastgele göründüğü için doğru nonce'u bulmak tamamen deneme işidir.
  - Ağ zorluğu (difficulty) otomatik ayarlar: Bitcoin'de bir blok yaklaşık 10 dakikada bulunacak biçimde ayarlanır ve 2016 blokta bir kontrol edilir. Bloklar hızlı bulunuyorsa istenen baştaki sıfır sayısı artırılır.
  - Madencinin teşviki blok ödülü (sunum anında 6,25 BTC) ve işlem ücretleridir (fee). Proof of Work çok enerji harcar; sunumda Bitcoin'in toplam enerji tüketiminin Türkiye'nin tüketiminden fazla olduğu iddia edildi.
  - **Ders sonrası not:** Sunumda Keccak'ın "SHA-256'nın değiştirilmiş bir sürümü" olduğu söylendi. Keccak, SHA-3 yarışmasını kazanan ve SHA-3'ün temelini oluşturan algoritmadır; SHA-2 ailesinden değildir. Ethereum büyük olasılıkla Keccak-256 kullanır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.